A REPORT

ON

# FindLink: Lost and Found

*An Android Application for Reporting and Recovering Lost Items*

*Submitted by*

Soyon Chandra Das
Roll No: 2307116

*In*

*Computer Science and Engineering*
*Second Year Second Term*




**Khulna University of Engineering & Technology**
**Course No: CSE-2200**
**Course Title: Advanced Programming Laboratory**

*Submitted to*

*Ehsanul Karim*
*-Lecturer*
*Dept. of Computer Science and Engineering*
*Khulna University of Engineering & Technology*

*Safin Ahmed*
*-Assistant Professor*
*Dept. of Computer Science and Engineering*
*Khulna University of Engineering & Technology*

# 1. Overview & Objectives

## 1.1 Purpose

FindLink is an Android application that reconnects people with the things they lose. A student or staff member can report a lost item, a finder can report a found item, and the two sides can talk directly inside the app. When ownership is claimed, the app verifies the claimant with a code that it issues itself, so an item is only handed over to the person who can prove it belongs to them.

The application is built for a campus environment where hundreds of items — ID cards, wallets, keys, bottles, umbrellas — change hands every semester, and most of them are never returned because there is no shared record of who found what.

## 1.2 Problem statement

- Lost-and-found notices are scattered across notice boards, Facebook groups and messaging apps, so a finder and an owner rarely see each other's posts.
- Anyone can claim a found item; there is no way to check whether the claimant is the real owner before handing it over.
- When the owner cannot be found at all, the finder has no formal place to deposit the item and no record of the handover.

## 1.3 Objectives

- Provide a single searchable feed of lost and found reports, filterable by type and category.
- Enable real-time, two-way messaging between the finder and the owner for every item.
- Verify ownership with an app-issued 6-digit security code that is never typed by the user at post time.
- Offer a formal deposit route through a lost & found desk, with a receipt code and office details.

## 1.4 Scope and target users

The current release targets students and campus staff. It supports email/password accounts, image attachments, chat, ownership verification and office deposits. It runs on Android 7.0 and above and was tested on a Xiaomi 12S running Android 15.

# 2. System Architecture & Technology Stack

FindLink follows a three-layer structure. Activities and fragments render the interface, small repository/util classes hold the business rules, and Firebase provides authentication and a real-time document database.

| Layer | Responsibility | Key classes |
|---|---|---|
| Presentation | Screens, fragments, adapters, layouts | MainActivity, ChatActivity, ItemDetailsActivity, HomeFragment, SearchFragment, ChatAdapter |
| Domain / rules | Ownership verification, chat ids, code generation, uploads | ClaimVerifier, SecurityCode, ClaimCode, ChatRepository, ImgbbUploader |
| Data | Authentication, documents, real-time listeners | FirebaseAuth, Cloud Firestore (users, items, chats, messages) |

## 2.1 Technology stack

| Component | Technology |
|---|---|
| Language / UI | Java, AndroidX, Material Components 3 |
| Minimum / target SDK | 24 / 35 (edge-to-edge on Android 15) |
| Authentication | Firebase Authentication — email & password |
| Database | Cloud Firestore with snapshot listeners and transactions |
| Images | imgbb REST API (upload) + Glide (display) |
| Camera | ActivityResultContracts.TakePicture with FileProvider |
| Gallery | ActivityResultContracts.GetContent |
| Build | Gradle 8, google-services plugin, Firebase BOM |

## 2.2 Navigation model

The app opens on a splash screen, then a four-tab home (Home, Search, Post, Profile) hosted by MainActivity. Fragments are added once and shown/hidden by tag so list position and scroll state survive tab switches. Chat, item details, claim, office deposit and inbox are separate activities launched with explicit intents.

# 3. Feature Walkthrough

Each feature below is shown with a screenshot captured from the running application on the test device.

## 3.1 Authentication

Users register with a name, email and password. Firebase Authentication creates the account, and a matching document is written to the users collection so the name and email can be shown everywhere in the app. On every launch the splash screen checks the session, so a logged-in user resumes directly at the home screen.

## 3.2 Home dashboard

The home screen is a live dashboard rather than a static menu. It greets the signed-in user, shows unread messages, offers a search shortcut, displays live counts of lost, found and deposited items, provides four quick actions and category filters, and lists the reports waiting at lost & found desks.

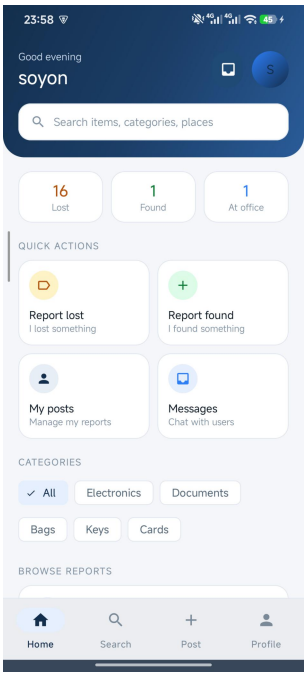



*Figure 3.1 — Home dashboard with live statistics and quick actions*

## 3.3 Posting a report

A single form handles both report types. The user chooses Lost or Found, enters the item name, category, location, contact number and a description that doubles as a verification question, and may attach a photo. Tapping the photo card opens a Material dialog with two choices — Camera or Gallery — so the user is never thrown straight into the gallery.

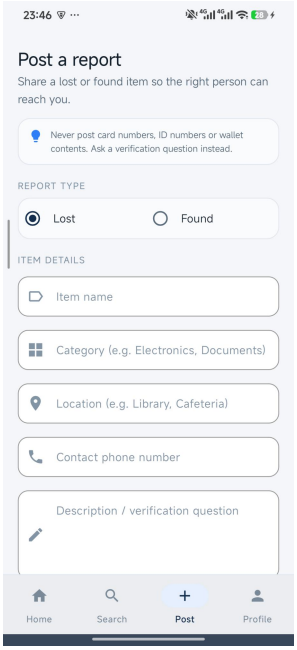



*Figure 3.2 — Post form*

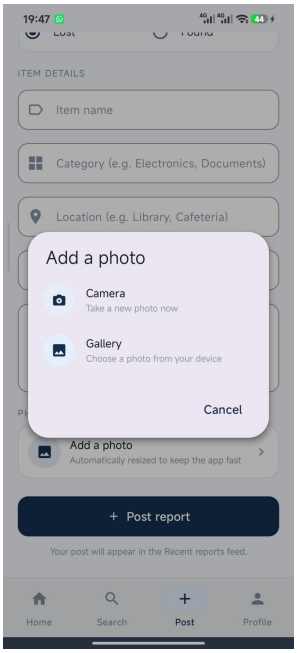



*Figure 3.3 — Camera / gallery chooser*

For a Lost report the app immediately generates a six-digit code and shows it once in a dialog; the same code is always available later on the item's details screen. Nothing about the code is typed by the user, which removes the weakest link in a manual scheme.

## 3.4 Search with type filters

Search matches the title, category and location of every report. Four filter chips — All, Lost, Found and At office — let a user browse a whole set without typing anything, and typing narrows the same filtered set. Results are sorted newest first and the counter states exactly what is being shown.

## 3.5 Item details

The details screen shows the photo, category chip, location, contact row (tap to dial), description and the report type. The action button changes with context: a visitor can message the finder or claim a found item, while the owner sees their security code and office options.

## 3.6 Real-time chat and inbox

Every item owns one conversation per pair of users, identified by a deterministic chat id. Messages are written to a sub-collection and both sides listen with a snapshot listener, so a reply appears without refreshing. Sent and received bubbles are styled differently, the last message is marked Seen when the other side reads it, and the inbox lists threads with unread dots. Opening a thread from the inbox passes the exact chat id, which guarantees both participants see the same conversation.

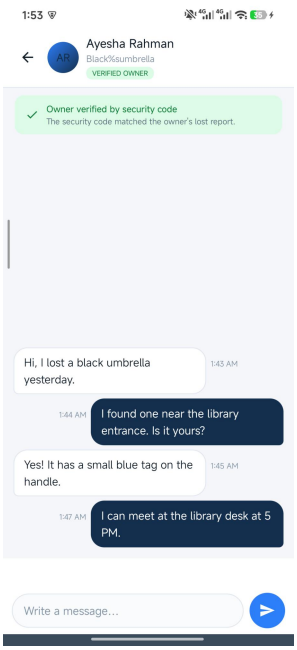



*Figure 3.4 — Conversation with verified-owner badge*

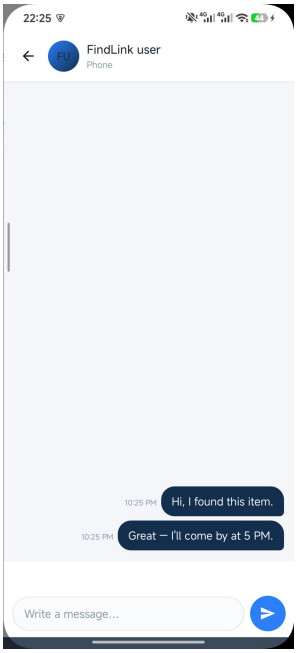



*Figure 3.5 — Inbox thread list*

## 3.7 Ownership verification

Ownership is proved with a code that the app issues, never one the user invents. When someone finds an item and opens the owner's lost report, the chat shows an amber banner: the owner is not verified yet. The finder asks for the code, enters it, and the app recomputes a salted SHA-256 hash per lost report and compares it with the stored hash. A match turns the banner green and adds a VERIFIED OWNER chip for both participants; a mismatch leaves the claim unverified, so the finder knows not to hand the item over.

The same rule is applied on the claim screen of a found report: the claimant types the code the finder gave them, and the claim message is marked verified only when the hash matches.

## 3.8 Lost & found office deposit

A finder who cannot trace the owner can deposit the item at one of four desks — Campus Security, Student Affairs, the Police Lost & Found desk or the City Lost Property Office. The app stores the chosen office, its address and opening hours, the deposit time and a receipt code such as FL-4821KM, then shows a receipt dialog. From that point the item leaves the active feed and appears under “Waiting at lost & found desks”, where the owner can find it and bring the code and an ID.

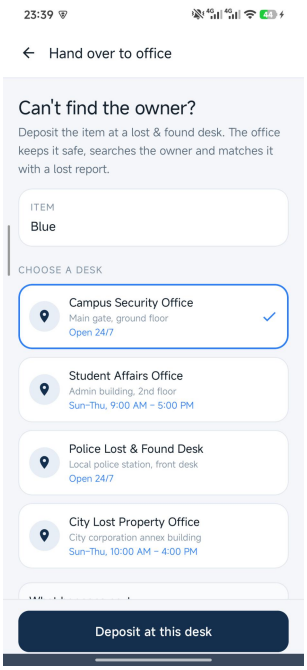



*Figure 3.6 — Choosing a lost & found desk*

# 4. External API Integration

## 4.1 imgbb image upload API

Photos are uploaded to imgbb through its REST API and only the returned URL is stored in Firestore. The app resizes the bitmap to a maximum dimension of 1080 pixels, compresses it to JPEG at 70% quality, encodes it as Base64 and sends a multipart/form-data POST with HttpURLConnection.

| Property | Value |
|---|---|
| Endpoint | POST https://api.imgbb.com/1/upload |
| Authentication | API key passed as a query parameter |
| Request body | multipart/form-data — field name image (Base64 string) |
| Response fields used | data.url (full image), data.thumb.url, success, status |
| Timeouts | 15 s connect, 30 s read; retry surfaced to the user as a toast |

The screenshot below is the real response captured from the API during development. The upload succeeded (success: true, status: 200) and returned a public URL that the app stores and Glide renders.



```
{
  "data":{
  "id":"cKG6ZrzL",
  "title":"t2",
  "url_viewer":"https:\/\/ibb.co\/cKG6ZrzL",
  "url":"https:\/\/i.ibb.co\/gb5SfRsV\/t2.png",
  "display_url":"https:\/\/i.ibb.co\/CpkbYHx2\/t2.png",
  "width":1080,
  "height":2400,
  "size":19159,
  "time":1794381104,
  "expiration":0,
  "image":{
  "filename":"t2.png",
  "name":"t2",
  "mime":"image\/png",
  "extension":"png",
  "url":"https:\/\/i.ibb.co\/gb5SfRsV\/t2.png"},
  "thumb":{
  "filename":"t2.png",
  "name":"t2",
  "mime":"image\/png",
  "extension":"png",
  "url":"https:\/\/i.ibb.co\/cKG6ZrzL\/t2.png"},
  "medium":{
  "filename":"t2.png",
  "name":"t2",
  "mime":"image\/png",
  "extension":"png",
  "url":"https:\/\/i.ibb.co\/CpkbYHx2\/t2.png"},
  "delete_url":"https:\/\/ibb.co\/cKG6ZrzL\/5c11ab7d5a556f44750ccf700fb9e8dc"},
  "success":true,
  "status":200}
```


*Figure 4.1 — Live JSON response from the imgbb upload API*

## 4.2 Why an external image host

Firebase Storage now requires a billing-enabled (Blaze) project for new deployments. To keep the project free while still storing photos, imgbb was chosen: it offers a simple HTTP API, no SDK, and a CDN URL that Glide can load directly. The trade-off is that images are public and depend on a third-party quota; the abstraction is isolated in ImgbbUploader, so switching to Firebase Storage later means changing one class.

# 5. Database & Firebase Configuration

## 5.1 Firebase Authentication

Email/password authentication is enabled in the Firebase console. Registration calls createUserWithEmailAndPassword; login calls signInWithEmailAndPassword; logout clears the session and the app returns to the login screen. A guard in MainActivity also redirects to login if no session exists, so a signed-out user can never see the home screen.

| Firebase setting | Value used in this project |
|---|---|
| Authentication provider | Email / password (enabled in Firebase Console → Authentication → Sign-in method) |

| Firebase setting | Value used in this project |
|---|---|
| Account records | Created by the app; profile name and email mirrored to users/{uid} |
| Database | Cloud Firestore, native mode, single region |
| Realtime Database | Not enabled — Firestore chosen for document queries and array-contains |
| Cloud Storage | Not enabled — images hosted on imgbb (see section 4.2) |
| Security rules | Authenticated access during development; production rules described in 5.4 |

## 5.2 Cloud Firestore data model

| Collection | Document id | Main fields | Used by |
|---|---|---|---|
| users | uid | name, email, userId, createdAt | Profile, chat names |
| items | auto id | itemId, userId, title, category, location, description, reportType, status, phone, timestamp, imageUrl, securityCode, securityHash, claimCode, office* | Home, Search, Details, Claim, Office |
| chats | itemId__uidA__uidB | chatId, itemId, itemTitle, postOwnerId, participants[], participantNames{}, lastMessage, lastMessageAt, lastRead{}, verifiedOwnerId | Inbox, Chat |
| chats/{id}/messages | auto id | senderId, senderName, text, createdAt, type, verified | Chat thread |

## 5.3 Queries and real-time listeners

- items where status = ACTIVE, and where status = AT_OFFICE, for the dashboard counters and feeds.
- items where userId = current uid, for “My posts” and for security-code verification.
- chats where participants array-contains current uid, ordered in memory by lastMessageAt.
- chats/{id}/messages ordered by createdAt, consumed through addSnapshotListener so both users update instantly.
- A Firestore transaction creates the chat document only once, and a WriteBatch writes a message and the chat summary atomically.

## 5.4 Security rules

During development the rules are open to authenticated clients so the demo works without a backend; the intended production rule set restricts every collection to signed-in users and limits writes to the document owner.

```
match /items/{id} { allow read: if request.auth != null; allow write: if request.auth.uid == resource.data.userId; }
```

## 5.5 Realtime Database and file storage

Cloud Firestore was selected over the Realtime Database because the data is document-shaped and needs compound queries plus array-contains. The Realtime Database is therefore not enabled for this release; if a live “online now” indicator is added later, it will use RTDB because presence is a small, fast-changing tree. Firebase Storage is not used because images are hosted on imgbb, as explained in section 4.2.

# 6. Testing & Verification

The application was built and installed on a Xiaomi 12S (Android 15) with every screen checked visually. Business rules were verified with instrumented tests that run against the real Firebase project, so they exercise the same documents the app uses.

| Test | What it proves | Result |
|---|---|---|
| SecurityCodeTest | 4–6 digit validation, salted hashing per item, correct code true / wrong code false, verified claim persisted | Passed |
| ClaimVerifierTest | Handover code match, security-code match, and no-match handling | Passed |
| OwnerVerifyTest | In-chat verification: correct code true, wrong code false, other user's code false, badge saved | Passed |
| ChatImprovementTest | Atomic message + summary, chat re-open keeps last message, unread clears after read, verified flag stored | Passed |
| OfficeDepositTest | Deposit code format FL-0000AA, near-unique generation, item flips to AT_OFFICE with office fields | Passed |
| WipeAllDataTest | Data reset routine deletes users, items, chats (with messages) and confirms empty collections | Passed |

## 6.1 Manual checks

- Keyboard opening on the post form no longer produces a blank screen after the IME inset fix.
- Headers clear the status bar on Android 15 edge-to-edge devices.
- Camera capture works through FileProvider and the photo appears in the post.
- A found-item chat shows no verification banner, by design; a lost-item chat does.

# 7. Challenges & Solutions

## 7.1 Unreadable text across the app

The first build mixed a DayNight theme with hard-coded colours, so dark text landed on dark surfaces. One light palette is now forced for values and values-night, with colours centralised in colors.xml.

## 7.2 Android 15 edge-to-edge headers

Targeting SDK 35 makes edge-to-edge mandatory, so headers slid under the status bar. WindowInsetsUtil now applies the top inset per header; an earlier attempt that also padded for the keyboard caused a white screen from a double IME inset.

## 7.3 Messages that never arrived

The message and the chat summary were written in two separate operations, so a failed summary update left the inbox stale, and two users opening a chat at the same time could overwrite each other's summary. They are now written in one WriteBatch, and chat creation runs inside a transaction.

## 7.4 “FindLink user” everywhere

Accounts created before the profile document existed had no name to show. A UserProfileSync helper now guarantees a users document with a name and email, deriving a readable name from the email prefix when the account never stored one.

### 7.5 Verifying an owner who never posted

A claimant with no lost report has no code to prove anything. The code is now issued by the app: every lost report receives one at creation, and the finder checks it inside the chat. Claimants without a report are told to ask the finder, and cannot become verified without a matching code.

## 8. Limitations & Future Work

### 8.1 Current limitations

- No push notifications: a message is visible only when the app is opened.
- Security codes are verified on the client; a production build should move the comparison to Cloud Functions or tighten the rules.
- The lost & found desk list is fixed in code, and there is no office-side dashboard to mark an item as collected.
- Message ordering uses the device clock, so two phones with very different times could show messages out of order.

### 8.2 Planned improvements

- Firebase Cloud Messaging for message and claim notifications.
- An office account role that can receive deposits and confirm collection.
- Server timestamps (FieldValue.serverTimestamp) for ordering, plus pagination for long threads.
- An image moderation step and a size/quota guard before upload.

## 9. Conclusion

FindLink turns a scattered, trust-based process into a recorded one. Lost and found reports live in a single searchable feed, two people can talk in real time about a specific item, and ownership is proved with a code the app itself issues — not one the claimant invents. When nobody can be found, the item is deposited at a desk with a receipt code and the owner can locate it from the same app.

The project also became a practical tour of Android fundamentals: activity lifecycle, explicit and implicit intents, click handling with listeners, multiple layout types, RecyclerView with custom adapters, FileProvider for camera output, HTTP communication with an external API, and a Firebase backend with authentication, document modelling, real-time listeners, transactions and batched writes.

## References


- Android Developers — Project structure and app modules. https://developer.android.com/studio/projects
- Android Developers — The activity lifecycle. https://developer.android.com/guide/components/activities/activity-lifecycle
- Android Developers — Intents and intent filters. https://developer.android.com/guide/components/intents-filters
- Android Developers — Layouts and ViewGroups. https://developer.android.com/develop/ui/views/layout/declaring-layout
- Android Developers — Create dynamic lists with RecyclerView. https://developer.android.com/develop/ui/views/layout/recyclerview
- Android Developers — Request runtime permissions. https://developer.android.com/training/permissions/requesting
- Android Developers — Connect to the network with HttpURLConnection. https://developer.android.com/develop/connectivity/network-ops/connecting
- Firebase — Authenticate with Firebase using email and password. https://firebase.google.com/docs/auth/android/password-auth
- Firebase — Cloud Firestore data model. https://firebase.google.com/docs/firestore/data-model
- Firebase — Get realtime updates with Cloud Firestore. https://firebase.google.com/docs/firestore/query-data/listen
- Firebase — Transactions and batched writes. https://firebase.google.com/docs/firestore/manage-data/transactions
- imgbb — API v1 documentation. https://api.imgbb.com/
- GeeksforGeeks — Android project folder structure. https://www.geeksforgeeks.org/android/android-project-folder-structure/
- GeeksforGeeks — Activity lifecycle in Android with demo app. https://www.geeksforgeeks.org/android/activity-lifecycle-in-android-with-demo-app/
- GeeksforGeeks — What is Intent in Android. https://www.geeksforgeeks.org/android/what-is-intent-in-android/
- GeeksforGeeks — Handling click events in Android. https://www.geeksforgeeks.org/android/handling-click-events-button-android-java/
- GeeksforGeeks — Firebase Realtime Database vs Firestore. https://www.geeksforgeeks.org/firebase/realtime-database-vs-firestore
- Material Design 3 — Components and theming. https://m3.material.io/